

UbuntuLink: Understanding Quantum Technology

1. Purpose of this document

This document explains quantum technology in simple terms and proposes a realistic way UbuntuLink can include it without turning the project into an impractical or overly complicated AI/quantum demonstration. The aim is to satisfy the quantum technology criterion while keeping the main product useful, understandable and deployable after the hackathon.

2. What is quantum technology?

Quantum technology uses principles from quantum physics to process information or perform measurements in ways that differ from ordinary computers. Traditional computers use bits that represent either 0 or 1. Quantum computers use quantum bits, called qubits, which can be prepared in combinations of 0 and 1 called superposition. Qubits can also be connected through a property called entanglement. These properties do not mean that quantum computers automatically solve every problem faster. Quantum algorithms are designed for particular types of problems, and current quantum hardware is still limited by noise, access restrictions and the number of usable qubits.

3. What does this mean for UbuntuLink?

UbuntuLink is a local services marketplace. Customers describe a job, find suitable providers, compare prices and availability, request a quote and confirm a booking. The quantum component should support one small decision-making problem within this workflow rather than replace the entire platform.

Our proposed use case is quantum-enhanced provider matching. The ordinary application will first filter providers using practical rules such as service type, service area and availability. A small quantum optimisation experiment will then demonstrate how several possible providers can be compared using factors such as distance, price, rating and urgency.

4. Recommended approach

I recommend using Qiskit as the quantum framework and Qiskit Aer as the simulator. Qiskit is a Python-based framework for creating, testing and executing quantum circuits. Qiskit Aer can run circuits in a simulated environment, allowing the team to develop and test without immediately depending on physical quantum hardware.

5. How the proposed solution would work

1. **Step 1: Customer request** – The customer describes a job, for example: “My kitchen tap is leaking.”
2. **Step 2: Service identification** – The application identifies or confirms the service category, such as plumbing.
3. **Step 3: Practical filtering** – The backend filters providers by service, service area, availability and basic eligibility.

4. **Step 4: Build a small optimisation problem** – A small set of eligible providers is scored using distance, estimated price, rating, experience and urgency.
5. **Step 5: Quantum experiment** – The selected provider options are converted into a simplified optimisation problem and tested with a Qiskit circuit.
6. **Step 6: Present the result** – The frontend displays recommended providers and explains the factors used. The quantum result is treated as an experimental supporting result, not as an unquestionable decision.

6. Example of a simplified problem

Assume three plumbers are available for a customer. The system must choose the most suitable option based on a weighted objective. A simplified score could consider:

- Distance: shorter travel distance is preferred.
- Price: a quote closer to the customer's budget is preferred.
- Rating: providers with stronger verified customer feedback receive a higher score.
- Availability: providers available within the requested time receive a higher score.
- Urgency: providers who accept urgent jobs may be prioritised for emergency requests.

For a first prototype, the team should keep the problem small, for example choosing one provider from three or four candidates. The purpose is to demonstrate the quantum workflow clearly, not to claim that a quantum computer is already better than a conventional matching algorithm.

7. The five quantum technology criteria

	How UbuntuLink can address it	Status / limitation
Working quantum circuit	Create and run a small Qiskit circuit for the simplified provider-selection problem.	Achievable in the prototype.
Quantum simulator	Run the circuit using Qiskit Aer and record the output.	Achievable without physical hardware.
Specific framework	Use Qiskit and document the relevant libraries and circuit approach.	Clearly identifiable.
Written explanation	Explain the problem, variables, circuit, results and limitations in the report and presentation.	Achievable.
Actual quantum hardware	Run the same or a smaller circuit on an accessible real quantum backend, if access is available.	Not guaranteed, depends on account access, backend availability, queue time and technical limits.

8. Physical hardware: what we must not claim

A simulator is not the same as a physical quantum computer. If the team only runs Qiskit Aer, we should state honestly that the circuit was simulated. To cover the hardware criterion, we would need to submit the circuit to a real quantum backend through a supported provider, subject to access and availability.

If physical hardware access is unavailable, we should still present the working circuit, simulator results, written explanation and Qiskit implementation. We should then explain that real-hardware execution is a planned extension rather than pretending it was completed.

9. Suggested technical architecture

- Python quantum service: a small Qiskit module or separate endpoint that receives a limited provider dataset.
- Qiskit: constructs the quantum circuit or optimisation experiment.
- Qiskit Aer: runs the circuit in simulation.
- Optional real backend: executes a reduced circuit on physical quantum hardware if access is approved.
- Result handler: returns the experimental output together with the classical scores and a clear explanation.

10. Important design decision: keep the main marketplace classical

The core UbuntuLink marketplace should continue to use conventional software for reliability and speed. Provider filtering, authentication, bookings, payments, messaging and safety workflows do not need to depend on a quantum computer. The quantum feature should be modular and optional.

This approach makes business sense because customers can still use UbuntuLink if the quantum service is unavailable, slow or being tested. It also prevents the project from becoming dependent on experimental technology for essential transactions.

11. Suggested implementation plan

- Confirm the judging requirement: ask whether a simulator is accepted, whether physical hardware is compulsory, and whether a working circuit must be demonstrated live.
- Select a very small problem: choose one provider from three or four candidates.
- Create a classical baseline first so the team has a reliable comparison.
- Install and test Qiskit and Qiskit Aer in a separate Python environment.
- Build and run the circuit locally using simulator data.
- Record the inputs, circuit approach, outputs and limitations.
- Investigate real hardware access and execute a reduced circuit only if access is available.
- Connect the experiment to the backend or create a controlled demo endpoint.
- Add a clear explanation to the frontend and presentation.
- Test the complete customer journey independently of the quantum feature.

16. Final recommendation

The recommended approach is: Quantum-Enhanced Provider Matching using Qiskit and Qiskit Aer, with optional execution on real quantum hardware. This approach can directly address a working circuit, simulator, written explanation and a specific framework. Physical hardware requires separate access so it'll be there only if possible. The team should focus on a small, clearly explained quantum experiment. A modest quantum component that is honest, testable and connected to a genuine business problem will be more credible than a large, complicated system that cannot be explained or maintained.